Instructivo de Integración Zero Trust

Componentes: Ory Kratos (Identity), Ory Hydra (OAuth2/OIDC), Ory Keto (Zanzibar ReBAC), Apache APISIX (API Gateway).

---

1. FRONTEND: Autenticación y Gestión de Cuentas (Ory Kratos)

El Frontend ya no es responsable de gestionar la lógica de sesiones, encriptación de contraseñas ni validación de recuperación de cuentas. Todo el Identity Management se delega a **Ory Kratos**.

1.1 Flujos de Interfaz de Usuario (UI Flows)

Para ejecutar acciones de identidad, el Frontend debe redirigir al usuario (Browser Redirect) a los flujos nativos de Kratos. Kratos gestionará el formulario, los errores y la seguridad, y devolverá al usuario a la aplicación una vez finalizado.

- **Registro de Usuarios:** Redirigir a `https://api.ztest.click/kratos/self-service/registration/browser`
- **Inicio de Sesión (Login):** Redirigir a `https://api.ztest.click/kratos/self-service/login/browser`
- **Recuperación (Contraseña Olvidada):** Redirigir a `https://api.ztest.click/kratos/self-service/recovery/browser`
- **Gestión de Perfil (Cambio de clave, 2FA):** Redirigir a `https://api.ztest.click/kratos/self-service/settings/browser`

1.2 Lectura de Sesión Activa

Para saber quién es el usuario logueado actualmente (para mostrar su nombre en el Header, por ejemplo), el Frontend debe hacer una petición `GET` incluyendo las credenciales (Cookies).

- **Endpoint:** `GET https://api.ztest.click/kratos/sessions/whoami`

Validar únicamente en FirmaEC.
Firmado electrónicamente por:
VERÓNICA TASINCHANO
ENRIQUEZ

- **Respuesta (Payload):** Devolverá un JSON con el objeto `identity`. De aquí se extrae el UUID (`id`) y los datos del usuario (`traits.email`, `traits.name.first`, etc.).

---

2. BACKEND: Lógica de Negocio y Autorización ReBAC (Ory Keto)

Keto es el motor de permisos basado en relaciones (Zanzibar). **Keto no tiene una base de datos de usuarios ni empresas; solo guarda relaciones abstractas.** El Backend de Negocio es el "Fuente de la Verdad" de los datos (tablas relacionales) y es responsable de notificar a Keto cuando ocurre un cambio jerárquico.

2.1 Creación de Empresas y Asignación de Roles

Cuando un usuario crea una empresa o invita a un nuevo empleado en la plataforma, el Backend debe realizar dos acciones:

1. Insertar el registro en la base de datos de negocio.
2. Inmediatamente, enviar una petición a la API de Escritura de Keto para establecer la relación jerárquica.

Contrato de Integración hacia Keto (Write API):

- **Endpoint Interno (APISIX):** `PUT https://api.ztest.click/keto/admin/relation-tuples`
- **Body (JSON):**

  JSON

  ```
  {
    "namespace": "Company",
    "object": "id-de-la-empresa-123",
    "relation": "admin",  // o "ventas", "member", etc.
    "subject_id": "uuid-del-usuario-desde-kratos"
  }
  ```

2.2 Listado de Usuarios por Empresa

Para listar usuarios, **NO** se consulta a la infraestructura de Identidad. El Backend debe hacer un `SELECT` a su propia base de datos filtrando por `empresa_id` y devolver la lista al Frontend. La base de datos del backend debe tener una tabla relacional (ej. usuarios_empresa) donde guarda una copia básica del UUID, el Email y el Rol. Así, cuando el Frontend pide la lista, su Backend hace un simple SELECT * FROM usuarios_empresa WHERE empresa_id = 'X' y responde en 10 milisegundos. Kratos y Keto se usan **solo para autenticar y autorizar en el momento del acceso**, no para renderizar vistas.

---

3. APISIX: Protección de Endpoints y Edge Authorization

Los microservicios (como `/signer/`) deben estar desplegados en redes privadas y **no deben poseer lógica de validación de tokens**. APISIX actuará como el "Zero Trust Edge", bloqueando el tráfico no autorizado antes de que toque los servidores de negocio.

3.1 El Flujo de Validación

Cuando una petición llega a APISIX (Ej: `GET /signer/documentos`):

1. **Validación de Identidad (OIDC):** APISIX intercepta el JWT (Bearer Token) del Frontend y consulta al endpoint de introspección de **Ory Hydra**. Si el token expiró o es inválido, APISIX rechaza la petición con `401 Unauthorized`.
2. **Extracción del Sujeto:** Si es válido, APISIX extrae el `UUID` del token.
3. **Autorización (ReBAC):** APISIX ejecuta un script Lua (Pre-function) que consulta la Read API de **Ory Keto** para verificar si ese `UUID` tiene la relación exigida para la ruta solicitada.
   - *Ejemplo de validación interna en APISIX:* `GET /relation-tuples/check?namespace=Document&object=doc-123&relation=view&subject_id=<UUID>`

4. **Forwarding:** Si Keto aprueba, APISIX enruta la petición hacia el Backend interno (ej: `18.135.180.59:8081`). El Backend confía ciegamente en la petición porque proviene de APISIX.

3.2 Acciones requeridas por el equipo de Desarrollo

El equipo de backend solo debe proporcionar al equipo de Infraestructura el diccionario de rutas y las políticas requeridas. Ejemplo:

- Ruta `/signer/admin/*` -> Exigir relación `admin` en namespace `Company`.
- Ruta `/signer/ventas/*` -> Exigir relación `ventas` en namespace `Company`.

La implementación técnica del bloqueo en la capa de red es responsabilidad del Arquitecto de Plataforma vía APISIX.